

06 — Implementation Roadmap, Repository & Deployment

A practical step-by-step plan for actually building the project

Build guide for the Context-Aware Online Distress Severity Detection project.

1. Build Order

- Step 1 — Define the four labels and annotation rules.
- Step 2 — Obtain or construct an ethically usable dataset.
- Step 3 — Implement PII sanitization and dataset validation.
- Step 4 — Perform exploratory data analysis and class-distribution analysis.
- Step 5 — Build TF-IDF + Logistic Regression baseline.
- Step 6 — Build and fine-tune RoBERTa.
- Step 7 — Compare classical and transformer baselines.
- Step 8 — Introduce class weighting.
- Step 9 — Experiment with focal loss.
- Step 10 — Investigate domain-adapted models.
- Step 11 — Investigate ordinal classification.
- Step 12 — Perform detailed error analysis.
- Step 13 — Add calibration and temperature scaling.
- Step 14 — Add attention and attribution analysis.
- Step 15 — Select the final model based on evidence.
- Step 16 — Build a minimal inference API.
- Step 17 — Build a minimal demonstration UI.
- Step 18 — Document experiments, limitations, and responsible-use boundaries.

2. MVP

The MVP should be:

Dataset → sanitization → train/test split → TF-IDF baseline → RoBERTa → four-class prediction → Macro F1/per-class metrics → confusion matrix.

This proves the end-to-end ML pipeline before adding advanced research features.

3. MVP v2

Add class weighting, focal loss, and explicit severe-class recall optimization.

Run controlled experiments and document the trade-offs.

4. Advanced Version

Add a domain-adapted transformer, ordinal objective, calibration, temperature scaling, attention analysis, stronger attribution, and systematic error analysis.

5. Repository Structure

```
distress-severity-nlp/  
■■■ README.md  
■■■ LICENSE  
■■■ pyproject.toml  
■■■ requirements.txt
```

```

■■■ configs/
■ ■■■ baseline.yaml
■ ■■■ roberta.yaml
■ ■■■ domain_model.yaml
■■■ data/
■ ■■■ raw/
■ ■■■ interim/
■ ■■■ processed/
■ ■■■ README.md
■■■ notebooks/
■ ■■■ 01_data_exploration.ipynb
■ ■■■ 02_baseline.ipynb
■ ■■■ 03_transformer_experiments.ipynb
■ ■■■ 04_error_analysis.ipynb
■ ■■■ 05_calibration.ipynb
■■■ src/
■ ■■■ data/
■ ■ ■■■ ingestion.py
■ ■ ■■■ sanitization.py
■ ■ ■■■ preprocessing.py
■ ■ ■■■ splitting.py
■ ■■■ tokenization/
■ ■ ■■■ tokenizer.py
■ ■■■ models/
■ ■ ■■■ baseline.py
■ ■ ■■■ roberta.py
■ ■ ■■■ domain_model.py
■ ■ ■■■ ordinal.py
■ ■■■ training/
■ ■ ■■■ trainer.py
■ ■ ■■■ losses.py
■ ■ ■■■ callbacks.py
■ ■■■ evaluation/
■ ■ ■■■ metrics.py
■ ■ ■■■ calibration.py
■ ■ ■■■ confusion.py
■ ■ ■■■ error_analysis.py
■ ■■■ interpretability/
■ ■ ■■■ attention.py
■ ■ ■■■ attribution.py
■ ■■■ utils/
■ ■■■ config.py
■ ■■■ logging.py
■ ■■■ reproducibility.py
■■■ tests/
■ ■■■ test_sanitization.py
■ ■■■ test_tokenization.py
■ ■■■ test_dataset.py
■ ■■■ test_model.py
■■■ experiments/
■ ■■■ baseline/
■ ■■■ roberta/
■ ■■■ loss_functions/
■ ■■■ ordinal/
■ ■■■ calibration/
■■■ reports/
■ ■■■ figures/
■ ■■■ tables/
■ ■■■ results.md
■■■ app/
■■■ backend/
■■■ frontend/

```

6. Deployment Architecture

Once the model is validated, a practical demonstration can use:

```

React frontend
↓

```

```
FastAPI backend
↓
PyTorch inference service
↓
Transformer tokenizer + model
↓
Prediction + probabilities + uncertainty
```

7. Deployment Comes Last

Do not build the frontend first.

The model, evaluation protocol, and limitations should be stable before exposing predictions through an application.

For a portfolio demo, the UI can display a sample text, predicted severity distribution, calibrated confidence, and a clear disclaimer about the model's intended use.

8. Testing

Unit-test sanitization so prohibited patterns are actually removed.

Test tokenization on empty, long, malformed, slang-heavy, and Unicode-heavy text.

Test model input/output shapes.

Test that inference uses the same tokenizer and preprocessing as training.

Add regression tests around important preprocessing behavior.

9. Final Technical Stack

ML: Python, PyTorch, Hugging Face Transformers, Hugging Face Tokenizers, scikit-learn.

Data: Pandas, NumPy, Regex, optionally Hugging Face Datasets.

Visualization: Matplotlib.

Experiment tracking: optionally Weights & Biases or MLflow.

Backend: FastAPI.

Frontend: React.

Deployment: containerization/cloud can be added after the model is validated.

10. What Not to Claim

Do not claim clinical diagnosis.

Do not claim the model can reliably determine a person's mental-health state from one post.

Do not claim the 0.89 Macro F1 or 94% sensitivity unless reproduced and documented.

Do not claim attention maps are causal explanations.

Do not call the system multitask unless you actually implement multiple prediction objectives.

11. The Guiding Principle

```
DATA → BASELINE → EXPERIMENT → EVIDENCE → IMPROVEMENT → ERROR ANALYSIS → DEPLOYMENT
```

This order keeps the project focused on ML substance while still leaving room for strong software engineering and a polished portfolio demonstration.